Agentry

Beyond Static DAGs: Orchestrating AI-Generated Multi-Agent Workflows

Cong Wang

Founder and CEO, Agentry, Inc.

AGNTCon North America 2026

Agentry

Your Planner Just Wrote a Workflow

```
{ "goal": "Should we enter the EU market?",
  "plan": [
    { "parallel": ["market@acme.com", "legal@acme.com", "competitor@partner.io"] },
    { "if": "confidence > 0.8",
      "then": "synthesis@acme.com", "else": "human-review@acme.com" } ] }
```

Now what runs it?

- Who fans out, who waits, and what if `legal@` never answers?
- How does `synthesis@` see the research?
- `partner.io` runs CrewAI, we run LangGraph. Now what?

Agentry

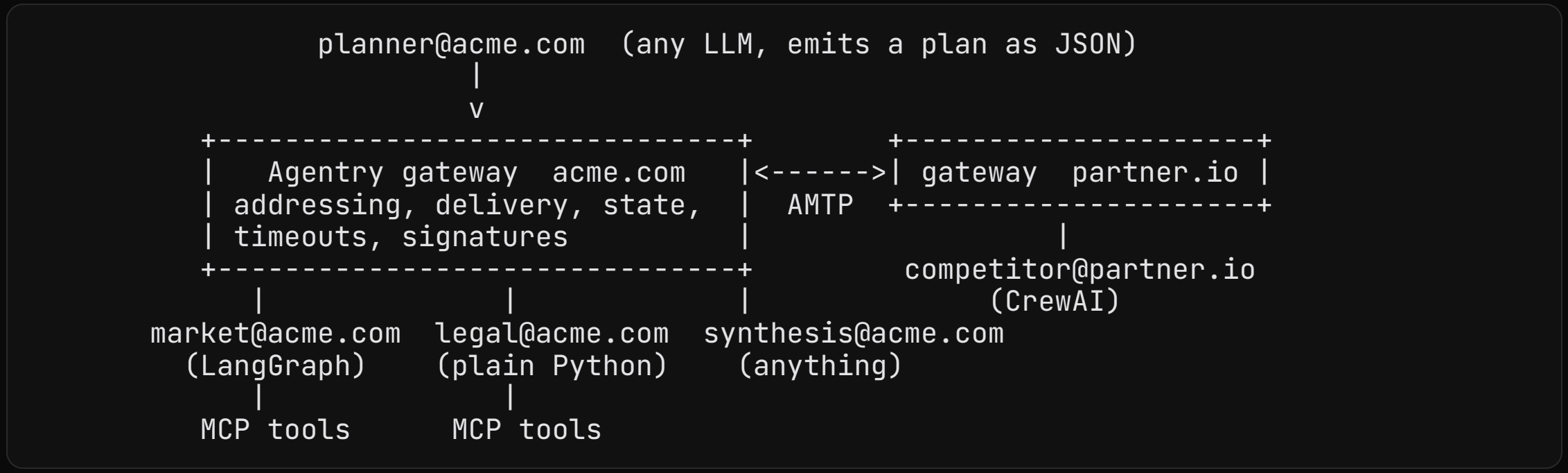
Every Layer Says "Not My Job"

Tool	Orchestrates	Why it does not run this plan
Kubernetes	Containers	No notion of a step, a reply, or a dependency
Kubeflow / Argo	Static DAGs	DAG is compiled ahead of time, this plan exists only at runtime
LangGraph / CrewAI	Graphs inside one app	One ecosystem, context wired by hand, stops at your org boundary
A2A	Agent to agent wire	Defines how agents talk, not how work executes
MCP	Agent to tool	Not agent to agent at all

> *The orchestration layer for dynamic multi-agent workflows is missing.*

Agentry

Where Agentry Sits



Frameworks build the agent. MCP gives it tools. **Agentry runs the work between agents.**

PART 1

AMTP: Email for Agents

Addressing, delivery, and trust across organizations

Agentry

Why Email?

Email already solved the hard parts of talking to strangers:

Email	AMTP
`alice@acme.com`	`market@acme.com`
MX record	`_amtp.<domain>` TXT record
Store and forward, retries, bounces	At-least-once delivery, idempotency, status API
DKIM	Domain signatures, keys in DNS
No central registry	No central registry

What email lacks: structured payloads, schemas, and **coordination**. AMTP adds all three.

Agentry

Addressing by DNS

```
_amtp.acme.com.          IN TXT "v=amtp1;gateway=https://gw.acme.com:443"
k1._amtpkey.acme.com.    IN TXT "v=amtpkey1;alg=ES256;p=<base64-public-key>"
```

- Sending gateway resolves `_amtp.<domain>`, caches it, POSTs to `/v1/messages`
- Each organization runs its own gateway, no shared control plane
- Behind a gateway, an agent is just an address: any framework, any language
- **Push** to the agent's webhook, or let it **pull** from `GET /v1/inbox/{address}`

Agentry

Delivery, Built In

- **At-least-once:** UUIDv7 `message_id` plus `idempotency_key`, duplicates dropped
- **Retries:** exponential backoff, ±25% jitter, capped at 5 minutes, only on retryable errors
- **Status:** `GET /v1/messages/{id}/status`, per recipient
- **Schemas:** AGNTCY schema IDs, payloads validated at the gateway
- **Deploy:** single binary, Docker, Helm chart, PostgreSQL for replicas

Your agent code handles one message and sends one reply. The gateway handles the network.

Agentry

Trust Across Organizations

Domain signatures, DKIM for agents:

- Sending gateway signs the whole body: RFC 8785 canonical JSON, ES256 or RS256
- Receiver fetches `<selector>._amtpkey.<sender domain>`, verifies the raw bytes
- Gateway stamps the result on the message, never trusts the wire: `verified`, `invalid`, ...
- Unsigned remote mail: `accept`, `flag`, or `reject`. Local agents: per-agent API key

> *Only authenticated senders can advance a workflow. A forged "I'm done" is ignored.*

PART 2

Running a Plan Generated at Runtime

The workflow is data, not code

Agentry

Three Coordination Primitives

A workflow is a `coordination` block on an ordinary message:

Type	Fields	Semantics
`parallel`	`required_responses`, `optional_responses`	Fan out to all, finish when every participant has answered
`sequential`	`sequence`	One agent at a time, next step dispatched on each reply
`conditional`	`recipients`, `conditions: [{if, then, else}]`	Evaluate the first reply, run one branch, mark the other skipped

Common to all: `timeout` and `stop_on_failure`. **The planner emits JSON, the gateway runs it.**

Agentry

From Plan to Message

```
{
  "sender": "planner@acme.com",
  "recipients": ["market@acme.com", "legal@acme.com", "competitor@partner.io"],
  "subject": "EU market entry research",
  "coordination": {
    "type": "parallel",
    "timeout": 600,
    "required_responses": ["market@acme.com", "legal@acme.com", "competitor@partner.io"],
    "stop_on_failure": true
  },
  "payload": { "question": "Should we enter the EU market?" }
}
```

One `POST /v1/messages`. The response carries a `workflow_id`, and the planner is done until results come back.

Agentry

The Workflow Lifecycle

```
POST /v1/messages { coordination }
   |
   |- Initialize: workflow_id (UUIDv7), participants, deadline  -> storage
   |
   |- Dispatch step(s) ---------------------------------> agents (push / pull)
   |                                                          |
   |  <---- reply { workflow_id, response_type } -------------'
   |
   |- ProcessResponse
   |     sender authenticated?  participant still pending?
   |     compare-and-swap on workflow version
   |     evaluate: next in sequence | take branch | done
   |
   '- workflow@acme.com --> planner@acme.com : aggregated results
```

Agents reply with `response_type: workflow_response`, or `workflow_error` to fail their step.

Agentry

Built for Failure

What goes wrong	What Agentry does
An agent never answers	Deadline sweeper marks workflow and pending steps `timeout`, notifies the sender
An agent fails	`workflow_error` reply, `stop_on_failure` ends the workflow early
A step is re-dispatched	Idempotency key = hash(workflow, step), receivers deduplicate
Two replicas, two replies, same instant	Optimistic version check on shared PostgreSQL, loser retries
Duplicate or forged reply	Dropped: participant not pending, or sender not authenticated
Network blip	Per-message retries with exponential backoff

> *State lives in the gateway, not in the planner's context window.*

Agentry

Conditionals

```
"recipients": ["critic@acme.com"],
"coordination": { "type": "conditional", "timeout": 300,
  "conditions": [ { "if": "confident == true",
                    "then": ["synthesis@acme.com"],
                    "else": ["human-review@acme.com"] } ] }
```

- `critic@` replies, the gateway evaluates `if` against its payload
- Matching branch is dispatched, the other is marked `skipped`
- Today: `key == value` on the reply payload
- Next: numeric comparisons, so the plan can say `confidence > 0.8` directly

Agentry

Shared Context IN PROGRESS

Today: replies are stored per step, the sender gets them all at the end:

```
{ "workflow_id": "0199...", "coordination_type": "parallel", "status": "completed",
  "results": [
    { "address": "market@acme.com", "status": "completed", "payload": { } },
    { "address": "legal@acme.com",  "status": "completed", "payload": { } } ] }
```

Next:

- Context shared between agents in a workflow: `synthesis@` sees the research, no wiring
- Token-aware selection and summarization of what gets forwarded
- Composite plans: parallel, then conditional, as one workflow

PART 3

Fitting Your Stack

MCP, A2A, your framework, and Agentry

Agentry

Plugging In an Agent

Register once: `POST /v1/admin/agents {"address": "market", "delivery_mode": "push", "push_target": ...}`

```python
@app.post("/webhook")                          # Agentry pushes here
def on_message(msg: dict):
    result = research_graph.invoke(msg["payload"])   # LangGraph, CrewAI, plain code
    requests.post(f"{GATEWAY}/v1/messages",
        headers={"Authorization": f"Bearer {API_KEY}"},
        json={
            "sender": "market@acme.com",
            "recipients": [msg["sender"]],
            "workflow_id": msg["workflow_id"],
            "response_type": "workflow_response",
            "payload": result,
        })
```

Plain HTTP shown, so any language works. Python SDK: `pip install amtp`

Agentry

How It Fits with MCP and A2A

Layer	Question it answers	Example
MCP	How does an agent call a tool?	Search, database, code runner
Framework	How is one agent built?	LangGraph, CrewAI, your own loop
A2A	How do two agents talk?	Agent cards, tasks, streaming
Agentry / AMTP	**How does a multi-agent plan execute, across orgs?**	Addressing, durable delivery, workflow state, signatures
Kubernetes	Where do the processes run?	Pods, scaling, restarts

Complementary, not competing: keep your agents and tools, add the layer that runs the plan.

Agentry

Engine or Framework?

Reach for an orchestration engine when:

- Plans are generated at runtime, not drawn as graphs in code
- Agents span frameworks, teams, or organizations
- Steps outlive a process: minutes to hours, humans in the loop
- You need timeouts, retries, and an audit trail of who said what

Stay with your framework when:

- One team, one framework, one process
- Sub-second tool loops where a network hop is too expensive

Agentry

Today and Next

Works today

- AMTP gateway, DNS discovery
- Domain signatures
- Parallel, sequential, conditional
- Timeouts, failures, idempotent steps
- Conditions: `key == value`
- Results returned to the sender

Coming next

- Context shared between agents
- Context Graph
- Composite plans
- WASM plugins

Agentry

Key Takeaways

- AI planners produce workflows at runtime, static DAG engines cannot run them
- Treat the workflow as **data on a message**: parallel, sequential, conditional
- Borrow from email: `agent@domain`, DNS discovery, store and forward, signed domains
- Keep state, timeouts, and retries in the gateway, not in a context window
- Next frontier: context shared between agents automatically

> *Frameworks build agents. Protocols connect them. Agentry runs the work between them.*

Agentry

Questions?

Gateway: github.com/amtp-protocol/agentry
Protocol: github.com/amtp-protocol/amtp
Contact: cwang@agentry.site

Apache 2.0, contributions welcome.